DOCUMENTO TÉCNICO PÚBLICO · v1.1 (revisão auditada)

AYORAI ATTRACTOR

Verificação auditável de respostas de inteligência artificial:
cada afirmação conferida contra a sua fonte,
cada veredito reproduzível.

Autor e direção técnica: Anderson Leon Ayora

Repositório: github.com/Ayorinha/ayorai-opensearch

Base deste documento: release v0.5.0 e branch main no commit ae44c23 (18/10/2026)

Revisão: v1.1, após auditoria de 18/10/2026; a v1.0 permanece no histórico do repositório (commit 9be3a38)

DOI (todas as versões): 10.5281/zenodo.23125082

Licença do código: Apache-2.0 · **Idioma:** português do Brasil



Este documento descreve arquitetura, princípios, funcionamento, rastreabilidade, aplicações e planos do projeto. Ele não é parecer jurídico, não é certificação e não declara resultados que o projeto ainda não obteve. Cada afirmação técnica relevante aponta para um arquivo do repositório ou para uma fonte pública listada nas Referências.

Sumário

1. Resumo executivo 3
2. Como este projeto nasceu 4
3. O problema que o projeto resolve 5
4. Diretrizes de desenho e por que foram escolhidas 6
5. Arquitetura 8
6. Como funciona: o Judge 2.0 passo a passo 10
7. Camada de stance: regras antes do modelo 15
8. Rastreabilidade: como tudo é registrado 16
9. Serviços: o que cada um faz e como é rastreado 19
10. Aplicações por setor 21
11. Processos judiciais e prova 23
12. Por que isso importa para a auditoria de IA no Brasil 25
13. Segurança e modelo de ameaças 26
14. Avaliação científica e estado atual dos resultados 27
15. Engenharia e governança 29
16. O que existe e o que ainda falta 30
17. Planos de melhoria contínua 31
18. Como participar, citar e usar 33
19. Glossário 34
20. Referências 36

COMO LER ESTE DOCUMENTO

Os capítulos 1 a 4 explicam o porquê do projeto e servem a qualquer leitor. Os capítulos 5 a 9 são técnicos e descrevem o funcionamento interno. Os capítulos 10 a 12 tratam do uso por setor e do contexto jurídico brasileiro. Os capítulos 13 a 17 tratam de segurança, evidência científica, estado atual e planos. Ao longo do texto, cada componente recebe um selo de estado: **IMPLEMENTADO** (no código e coberto por testes), **BIBLIOTECA** (existe e é testado, mas não está ligado à API pública), **PROPOSTO** (decidido ou desenhado, ainda não implementado) e **PENDENTE** (ainda não iniciado).

1 Resumo executivo

O **AYORAI ATTRACTOR** é um motor de código aberto para **verificar, afirmação por afirmação, se as respostas produzidas por sistemas de inteligência artificial são sustentadas pelas fontes que dizem usar**, e para registrar essa verificação de forma que qualquer pessoa possa reproduzi-la e conferi-la depois.

A ideia central é simples: um modelo de linguagem pode ajudar a analisar textos, mas **não pode ser o juiz da sua própria resposta**. No ATTRACTOR, a decisão final sobre cada afirmação é tomada por um componente determinístico (o *Judge*), que aplica regras explícitas, versionadas e identificadas por hash criptográfico. Para entradas estruturadas válidas do contrato r2-a (afirmação, átomos, evidências com posições, textos-fonte e stances explícitas), o Judge 2.0 produz o mesmo resultado canônico, byte a byte, sempre que a versão das regras for a mesma (teste T7). Essa garantia cobre o Judge, não o pipeline inteiro: extração de afirmações, NLI e tradução dependem de modelos e só são reprodutíveis se as versões forem fixadas e registradas.

5	1	1.057	4
vereditos possíveis por afirmação	autoridade de veredito: o Judge determinístico	testes aprovados localmente no commit ae44c23; CI verde no mesmo SHA	releases com DOI no Zenodo (v0.2.0 a v0.5.0)

O que o projeto entrega hoje

- Um **contrato formal de adjudicação** (Judge 2.0, ADR-009) com cinco vereditos, três estados de decisão, quatro níveis de robustez e regras de agregação para afirmações compostas.
- Uma **API HTTP** com rotas de pesquisa, auditoria operacional e verificação (`/v1/verify`), incluindo, no contrato r2-a, a conferência de que cada trecho citado existe nas posições declaradas do texto-fonte fornecido pelo chamador.
- Uma **trilha de prova** que liga afirmação, trecho, posição no documento, hash da fonte, versão das regras e hash do resultado, com serialização canônica RFC 8785.
- Uma **metodologia de avaliação** com pré-registro, conjuntos congelados por SHA-256 e divulgação explícita de limitações e resultados negativos.

O que o projeto ainda não entrega

- **Não existe ainda resultado de benchmark em português.** Os resultados publicados vêm de conjuntos sintéticos de desenvolvimento em inglês, com 30 casos.
- O **registro de auditoria criptograficamente verificável** (ADR-010) está decidido e tem a biblioteca da fase 1 integrada, mas não está implantado.
- A API **não extrai afirmações nem infere stance sozinha**: recebe esses dados já estruturados.
- O projeto **não é certificado**, não tem cliente em produção e não afirma parceria com nenhuma instituição.

> **EM UMA FRASE**
>
> O ATTRACTOR transforma “a IA disse, e citou uma fonte” em “este trecho, desta fonte, nesta posição, sustenta (ou contradiz) esta afirmação, segundo estas regras, e qualquer um pode recalcular o mesmo resultado”.

2 Como este projeto nasceu

O AYORAI ATTRACTOR foi criado por **Anderson Leon Ayora**. O primeiro commit do repositório data de **30 de setembro de 2026**, e a primeira versão marcada (tag v0.1.0) foi criada no mesmo dia.

A necessidade que originou o projeto está registrada no próprio problema que ele declara resolver: sistemas de IA generativa produzem respostas fluentes, com aparência de rigor e com citações, mas a fonte citada muitas vezes não sustenta o que foi afirmado. Para uma organização que precisa prestar contas, isso cria uma lacuna de auditoria. Quando alguém pergunta *por que* o sistema chegou a determinada conclusão, uma resposta narrativa não basta. É preciso mostrar qual trecho de qual documento sustenta cada afirmação e quem (ou que regra) decidiu isso.

Os documentos institucionais do projeto (`docs/INVESTIDORES.md`) registram a crítica que orienta o desenho: quando o mesmo modelo produz a resposta e também decide se ela está correta, funções que deveriam ser separadas ficam concentradas num componente que não é reproduzível nem explicável por regra perante um auditor, um perito ou um juiz. A decisão de desenho que distingue o projeto nasceu daí: **separar quem analisa de quem decide**, e fazer com que quem decide seja um conjunto de regras públicas, versionadas e verificáveis.

O contexto que tornou a necessidade evidente

Enquanto o projeto era construído, o problema aparecia com frequência crescente nos tribunais, no Brasil e no exterior:

- **Estados Unidos, 2023.** No caso *Mata v. Avianca*, a Justiça Federal de Nova York sancionou advogados que apresentaram precedentes inexistentes gerados por IA. É considerado o primeiro caso de grande repercussão do tipo.[1]
- **Estados Unidos, 2026.** No início de junho de 2026, a base de dados mantida pelo pesquisador Damien Charlotin listava 1.058 casos em tribunais americanos envolvendo uso indevido de IA generativa em peças processuais.[1]
- **Paraná.** A 9ª Câmara Cível do TJPR multou um advogado por litigância de má-fé por citar um precedente do STJ que não foi localizado no repositório oficial e que, segundo o relator, provavelmente foi gerado por IA. O caso foi encaminhado à OAB/PR.[2]
- **Paraíba.** Na comarca de Sousa, uma petição escondia comandos dirigidos a ferramentas de IA (*prompt injection*) para tentar manipular a análise judicial. O juízo aplicou duas multas que somam R$ 32,8 mil e encaminhou o caso à OAB e ao Ministério Público.[3]
- **Goiás, 2026.** Uma juíza determinou a emenda de uma petição inicial com sinais de ter sido gerada por IA sem revisão humana e comunicou a OAB.[4]

Esses casos mostram duas falhas que o ATTRACTOR ataca de frente. A primeira é a **citação que não sustenta a afirmação** (ou que nem existe). A segunda é o **texto de um documento tentando instruir a máquina**. No ATTRACTOR, uma citação só vale se o trecho for localizado no documento original, e o texto das evidências é sempre tratado como dado, nunca como instrução.

> **NOTA EDITORIAL**
> Os casos acima são contexto público e não foram analisados pelo ATTRACTOR. Eles ilustram a necessidade; não são resultados do sistema.

3 O problema que o projeto resolve

Uma resposta de IA costuma conter várias afirmações. A fonte citada pode sustentar uma, contradizer outra e não dizer nada sobre uma terceira. Quando a resposta é avaliada como um bloco único, essas diferenças desaparecem, e a presença de uma citação real dá credibilidade a afirmações que ela não sustenta.

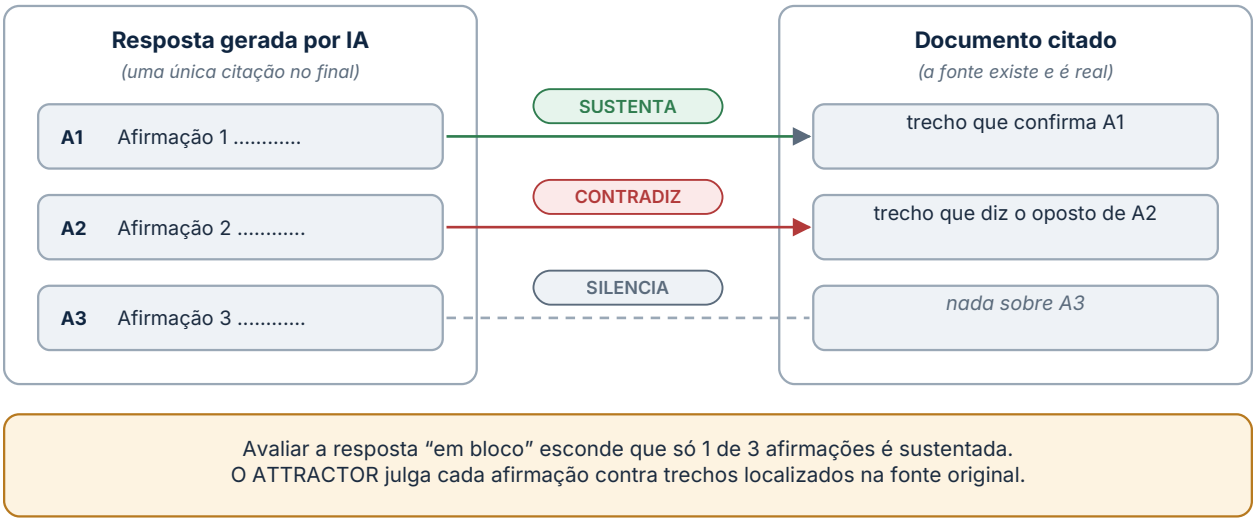


Figura 1. A existência da fonte não basta. É preciso verificar a relação entre cada afirmação e um trecho localizado da fonte.

Três perguntas que precisam de resposta auditável

1. **Esta afirmação é sustentada?** Não "a resposta é boa?", mas "esta frase específica tem apoio nestas evidências?".
2. **Por quê?** Qual trecho, de qual documento, em que posição, e qual regra levou à conclusão.
3. **Dá para conferir depois?** Outra pessoa, com os mesmos dados e a mesma versão das regras, chega ao mesmo resultado?

As abordagens mais comuns no mercado respondem bem à primeira pergunta em termos estatísticos, por exemplo com um escore de 0 a 1 de "groundedness" ou um segundo modelo atuando como juiz. Elas respondem mal à segunda e à terceira. O ATTRACTOR foi desenhado a partir da segunda e da terceira.

O que o ATTRACTOR não promete

- Não prova que a fonte é verdadeira, atual, completa ou juridicamente aplicável. O veredito vale *dentro do conjunto de evidências fornecido*.
- Não detecta toda alucinação possível.
- Não interpreta hierarquia normativa, vigência ou jurisprudência.
- Não substitui revisão humana, parecer jurídico ou certificação.

4 Diretrizes de desenho e por que foram escolhidas

Cada diretriz abaixo é uma decisão registrada em um ADR (*Architecture Decision Record*) no repositório, com data, contexto, alternativas consideradas e aprovação do mantenedor.

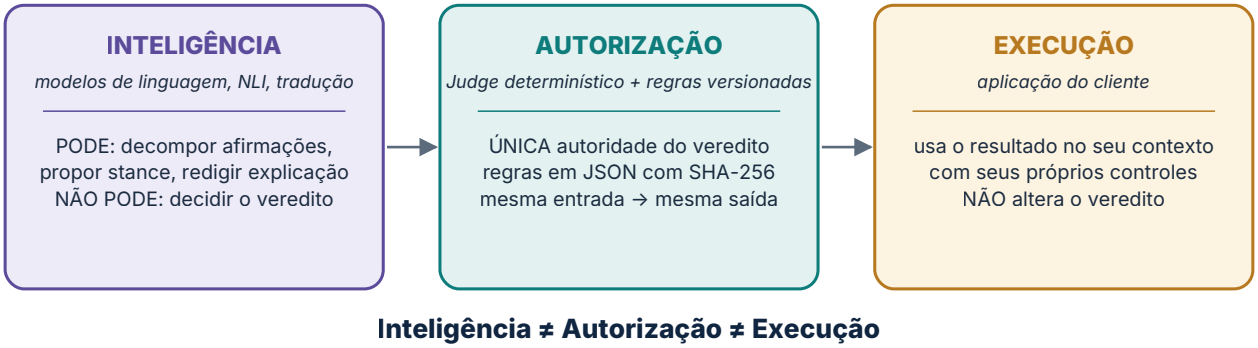


Figura 2. O princípio central: análise, autorização do veredito e execução são papéis separados.

Diretriz	O que significa	Por que foi escolhida	Registro
1. Inteligência ≠ Autorização ≠ Execução	Separar quem analisa (modelo), quem decide (Judge) e quem usa o resultado (aplicação).	Um texto fluente não é uma decisão. Misturar os papéis concentra poder em um componente que não é explicável nem reproduzível.	README, ADR-002, ADR-009
2. Judge determinístico como única autoridade	Só o Judge emite veredito. Testes de arquitetura (AST) restringem a construção do veredito fora do Judge; não provam todos os comportamentos em tempo de execução.	Reprodutibilidade e responsabilização: as regras são dados versionados, identificados por `rules_sha256`.	ADR-002, ADR-009
3. Afirmação como entrada	`verify(claims, documents)` recebe as afirmações do chamador; nunca deriva afirmações das evidências.	Impede que um documento "reescreva a pergunta" e reduz o risco de *prompt injection*.	ADR-004
4. Stance em três estados	SUPPORTS, CONTRADICTS ou NEUTRAL entre evidência e afirmação.	Ausência de apoio não é contradição; "não confirmado" e "contradito" são coisas diferentes.	ADR-003
5. Abster-se em vez de adivinhar	Estados ABSTAINED e INVALID_INPUT, sempre sem veredito. No r2-a, os motivos de abstenção são informados pelo chamador.	Um sistema auditável precisa dizer "não sei" ou "entrada inválida" em vez de inventar uma resposta.	ADR-009 §3–4

6. Conflito explícito	Veredito CONFLICTING quando há apoio e contradição independentes.	Não escolher silenciosamente a fonte “preferida” esconde informação do auditor.	ADR-009 §7
7. Evidência só conta se localizada	Offsets válidos, trecho idêntico ao texto-fonte normalizado (NFC) e hash conferido.	Bloqueia citações forjadas: a IA não pode “provar” com um trecho que não existe.	ADR-009 §6
8. Independência de fontes	O Judge conta clusters sem dependência conhecida, não documentos.	Dez republicações da mesma notícia não são dez confirmações.	ADR-006
9. Regras antes do modelo	Números, datas, unidades e negação são decididos por regras determinísticas antes de qualquer NLI.	Comparações exatas não devem depender de probabilidade.	ADR-005, ADR-006
10. Tudo versionado e com hash	Entrada, saída, regras e artefatos de avaliação têm SHA-256 com serialização canônica RFC 8785.	Permite detectar alteração e reproduzir resultados entre implementações e linguagens.	ADR-009 §10
11. Honestidade metodológica	Pré-registro, conjuntos congelados, divulgação de limitações e de resultados negativos.	Em auditoria, credibilidade vem de não esconder o que não funciona.	docs/eval/
12. Privacidade por padrão	Não persistir prompts brutos, trechos ou dados pessoais quando hashes bastam; compromissos com sal no log futuro.	Rastreabilidade não pode virar vazamento de dados (LGPD).	ADR-010, PROVENANCE-MODEL

5 Arquitetura

O sistema tem três camadas: a camada de serviço (API e linha de comando), o núcleo de verificação (Judge e regras) e a camada de rastreabilidade (hashes, trilha de auditoria e, no futuro, log de transparência).

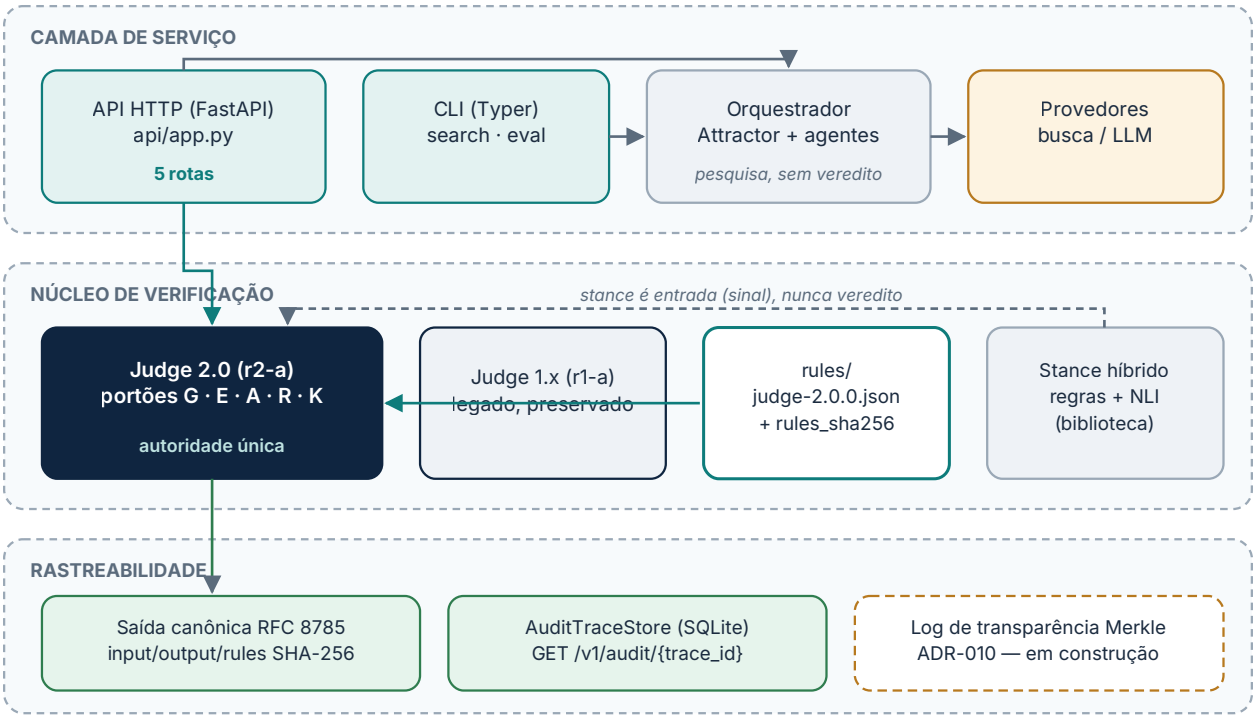


Figura 3. Visão em camadas. Setas sólidas são chamadas existentes no código; setas tracejadas indicam relações de dados ou componentes futuros.

Componente	Função	Estado
`api/app.py`	API FastAPI com as rotas `/health`, `/v1/opensearch`, `/v1/audit`, `/v1/audit/{trace_id}` e `/v1/verify`. Autenticação por chave de API ativa por padrão e com falha fechada se mal configurada; pode ser desligada explicitamente (`ATTRACTOR_AUTH_MODE=off`). `/health` é público. Há limite configurável de tamanho do corpo da requisição.	IMPLEMENTADO
`cli/main.py`	Linha de comando: `search` (pesquisa orquestrada) e `eval` (avaliação Golden).	IMPLEMENTADO
`orchestrator.py`	Orquestra agentes de pesquisa e provedores. Respostas operacionais saem com `verdict_authority = not_adjudicated`.	IMPLEMENTADO
`verification/judge2/`	Judge 2.0: portões G e E, regras A, R e K, saída canônica.	IMPLEMENTADO
`rules/judge-2.0.0.json`	Regras executáveis como dados versionados; as tabelas-verdade publicadas são geradas a partir deste arquivo.	IMPLEMENTADO

`verification/stance.py`, `nli.py`, `translation.py`	Detecção híbrida de stance (regras + NLI).	BIBLIOTECA
`verification/claim_pipeline.py`	Extração, evidências, stance e Judge no fluxo de avaliação Golden.	BIBLIOTECA
`auditlog/`	Árvore de Merkle RFC 6962, provas de inclusão e consistência, compromisso com sal (fase 1 da ADR-010).	BIBLIOTECA
Log de transparência implantado	Checkpoints assinados, testemunhas, gravação de `/v1/verify`.	PROPOSTO

LIMITE IMPORTANTE DA API ATUAL

A rota `/v1/verify` recebe afirmações, evidências e stances **já estruturadas** pelo cliente. Ela confere offsets, trechos e hashes (no contrato r2-a), mas não extrai afirmações nem infere stance sozinha. A extração e a inferência existem como biblioteca e são usadas no fluxo de avaliação. Integrá-las à API é parte do roteiro.

6 Como funciona: o Judge 2.0 passo a passo

O Judge 2.0 é definido pela ADR-009 e implementado no contrato `r2-a` da rota `/v1/verify`. Cada requisição avalia uma afirmação (simples ou composta) e recebe seus átomos, as evidências com posições no texto-fonte, os textos-fonte e as relações de stance. O resultado sai em nove etapas.

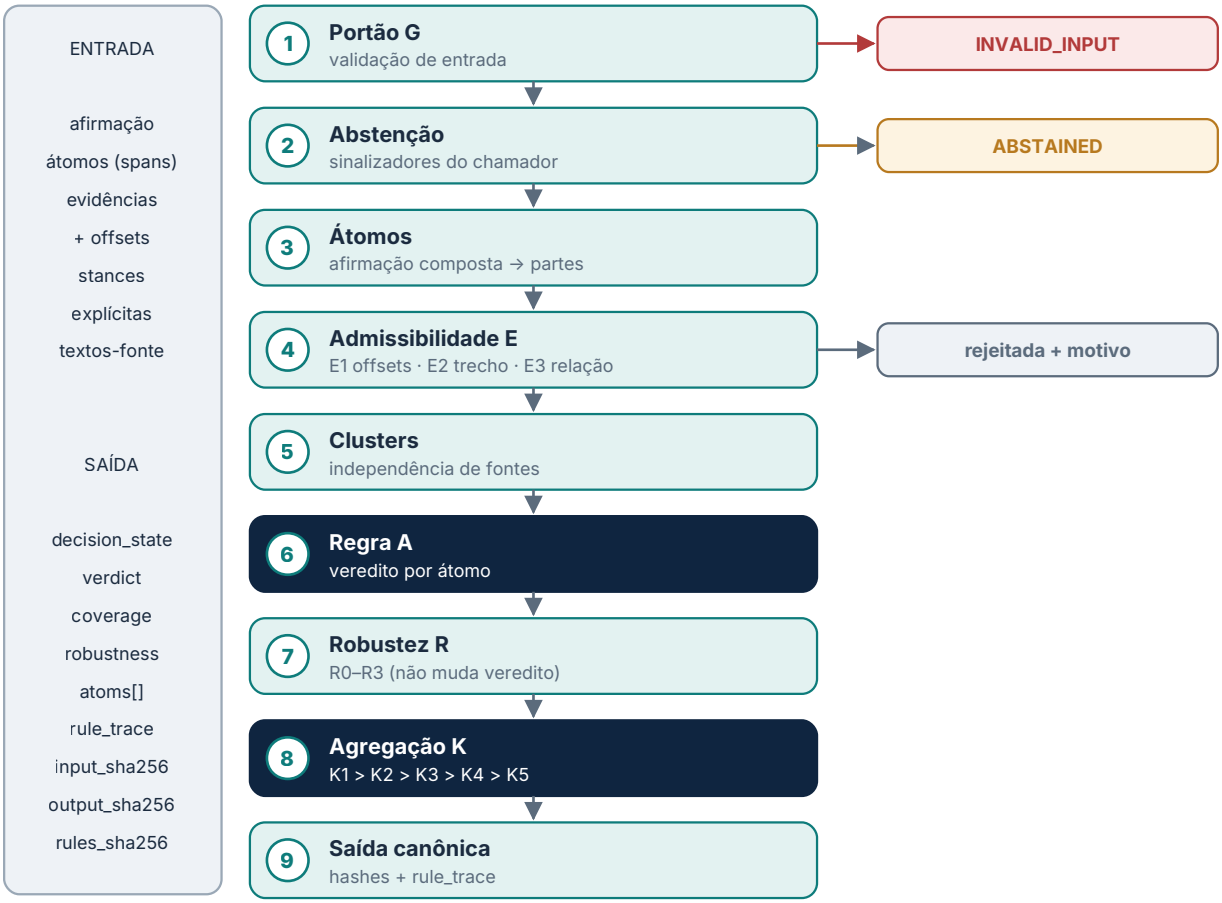


Figura 4. As nove etapas do Judge 2.0. As etapas escuras (regra A e agregação K) são as que efetivamente produzem o veredito.

Etapa 1 — Portão G: validação de entrada

Entradas malformadas não recebem veredito; recebem `INVALID_INPUT` com um código específico. Cada código tem um teste negativo próprio.

Código	Situação
`SCHEMA_VIOLATION`	entrada incompatível com o esquema declarado
`EMPTY_CLAIM`	afirmação vazia ou só com espaços
`NO_ATOMS`	nenhum átomo informado: no r2-a toda afirmação precisa de ao menos um átomo
`DUPLICATE_ID`	identificador repetido onde deveria ser único
`DANGLING_REFERENCE`	referência a afirmação, átomo, evidência ou stance inexistente

`DUPLICATE_STANCE`	mais de uma stance para a mesma relação
`ATOM_SPAN_OUT_OF_RANGE`	posições do átomo fora da afirmação, ou texto do átomo diferente do trecho declarado
`UNSUPPORTED_PROTOCOL_VERSION`	versão de protocolo não suportada

Etapa 2 — Abstenção

Quando não é legítimo decidir, o resultado é `ABSTAINED`, sem veredito. Os motivos são fora de escopo, afirmação não factual, falta de âncora temporal, falha de recuperação e falha de integridade do conjunto de evidências. No contrato r2-a, esses motivos chegam como sinalizadores informados pelo chamador (`out_of_scope`, `factual_claim`, `temporal_anchor_present`, `retrieval_succeeded`, `evidence_pool_integrity_ok`); o Judge não decide sozinho, por exemplo, se uma afirmação está fora de escopo. Abstenção não é veredito e não é contada como `UNVERIFIED`.

Etapa 3 — Decomposição em átomos

Uma afirmação composta (por exemplo, "o prazo é de 30 dias *e* pode ser prorrogado") é dividida em átomos, cada um apontando para um trecho exato da frase original, e o texto do átomo precisa coincidir com esse trecho. Uma afirmação simples tem um único átomo. A decomposição pode ser feita com ajuda de modelo, mas é registrada como artefato versionado. O Judge valida a estrutura da decomposição; a qualidade semântica dela é medida separadamente no benchmark.

Etapa 4 — Admissibilidade da evidência (E1, E2, E3)

Critério	Exigência	Se falhar
E1	O texto-fonte indicado foi fornecido; offsets válidos, ordenados e dentro do documento.	`SOURCE_UNAVAILABLE` ou `OFFSET_INVALID`
E2	O trecho confere com o texto-fonte normalizado em Unicode NFC e com o hash do conteúdo, quando fornecido.	`EXCERPT_MISMATCH`
E3	Existe relação explícita SUPPORTS ou CONTRADICTS com o átomo.	`NEUTRAL_STANCE`

Cada par evidência × átomo é avaliado separadamente; uma evidência sem stance declarada para aquele átomo é rejeitada como `NEUTRAL_STANCE`. A evidência rejeitada não entra na contagem e **sempre** aparece no resultado com o motivo. Nada é descartado em silêncio.

Etapa 5 — Clusters de independência

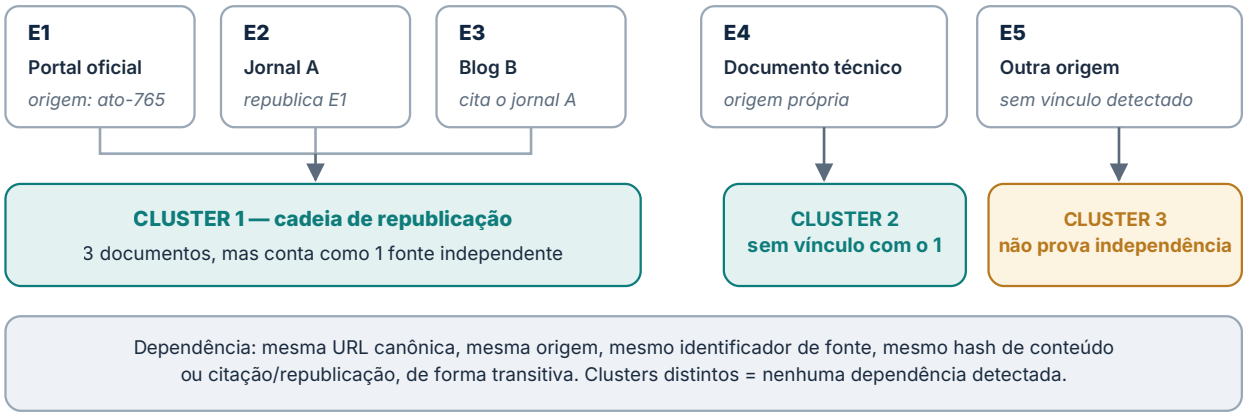


Figura 5. Republicações e cadeias de citação formam um único cluster. Clusters distintos significam que nenhuma dependência foi detectada, não que a independência foi provada.

Duas evidências ficam no mesmo cluster quando há dependência conhecida: mesma URL canônica, mesma origem, mesmo identificador de fonte, mesmo hash de conteúdo ou cadeia de citação/republicação, de forma transitiva (ADR-006). O contrato exige `origin_id` ou `canonical_url` em toda evidência. Evidências sem dependência conhecida ficam em clusters separados e contam separadamente em S e C; isso registra a **ausência de dependência detectada**, não uma independência comprovada.

Etapa 6 — Regra A: veredito de cada átomo

Sendo S o número de clusters admissíveis de suporte e C o de contradição (clusters sem dependência conhecida entre si):

Regra	Condição	Veredito do átomo
`A1`	S = 0 e C = 0	**UNVERIFIED**
`A2`	S ≥ 1 e C = 0	**SUPPORTED**
`A3`	S = 0 e C ≥ 1	**REFUTED**
`A4`	S ≥ 1 e C ≥ 1	**CONFLICTING**

Etapa 7 — Robustez R0 a R3

A robustez descreve a força da evidência **sem nunca alterar o veredito**. Ela é calculada separadamente para o lado que sustenta e para o lado que contradiz.

Nível	Significado
R0 — NONE	nenhum cluster decisório admissível
R1 — SINGLE_SOURCE	exatamente um cluster no lado avaliado
R2 — INDEPENDENCE_UNKNOWN	dois ou mais clusters, mas proveniência incompleta em alguma evidência
R3 — CORROBORATED	dois ou mais clusters, todos com proveniência completa

COMO LER A ESCALA DE ROBUSTEZ

A escala é uma ordem de informação, não uma medida de verdade. R2 fica acima de R1 porque há ao menos dois clusters sem dependência conhecida: no pior caso eles equivalem a uma única fonte (R1), no melhor a uma corroboração (R3). A diferença entre R2 e R3 depende do campo `provenance_complete` de cada evidência, que é **declarado pelo chamador** e vale `true` por padrão; o Judge confere offsets e trechos, mas não verifica sozinho essa completude. Por isso R3 significa "corroborado segundo a proveniência declarada". Qualquer mudança dessa semântica exige nova decisão versionada (ADR).

Etapa 8 — Agregação K para afirmações compostas

A precedência é aplicada nesta ordem, com pesos iguais entre átomos:

Regra	Condição	Veredito da afirmação
K1	algum átomo REFUTED	**REFUTED**
K2	senão, algum átomo CONFLICTING	**CONFLICTING**
K3	senão, todos SUPPORTED	**SUPPORTED**
K4	senão, algum SUPPORTED	**PARTIALLY_SUPPORTED**
K5	senão	**UNVERIFIED**

Uma contradição em qualquer parte impede declarar a afirmação como sustentada (K1). Um átomo isolado nunca recebe `PARTIALLY_SUPPORTED`; esse veredito é exclusivo da agregação.

Etapa 9 — Saída canônica

O resultado contém `judge_rules_version`, `rules_sha256`, `input_sha256`, `decision_state`, `reason_codes`, `verdict`, `coverage`, `robustness`, `atoms` (com evidências admitidas e excluídas) e `rule_trace`. O `output_sha256` é calculado sobre a serialização canônica RFC 8785 do resultado antes da inclusão do próprio campo.

Exemplo completo, executado na API

Afirmação hipotética: *"O regulamento X fixa prazo de 30 dias para resposta e permite prorrogação."* Fontes fictícias: o texto do "regulamento X" (doc1) e um resumo que cita esse regulamento (doc2). A norma não existe; o exemplo é didático. A requisição abaixo foi enviada à rota `POST /v1/verify` (r2-a) no commit ae44c23, e o resultado transcrito é o que a API devolveu.

```
doc1: "Art. 5º A prestadora deve responder em até 30 dias. Parágrafo único.
       O prazo não pode ser prorrogado."
doc2: "Resumo: segundo o regulamento X, a prestadora deve responder em até 30 dias."

átomo a1  "prazo de 30 dias para resposta"
  e1  doc1  "deve responder em até 30 dias"   origin_id=regulamento-x       SUPPORTS
  e2  doc2  "deve responder em até 30 dias"   cited_origin_id=regulamento-x SUPPORTS
      e2 cita a origem de e1  ->  mesmo cluster
  S = 1, C = 0  ->  2.0.0/A2  ->  SUPPORTED   robustez R1

átomo a2  "permite prorrogação"
  e3  doc1  "O prazo não pode ser prorrogado"                              CONTRADICTS
  S = 0, C = 1  ->  2.0.0/A3  ->  REFUTED     robustez R1

resultado: DECIDED  REFUTED (2.0.0/K1)  robustez R1
coverage: supported=1 refuted=1 conflicting=0 unverified=0 total=2
output_sha256 = a1a38fe0c3039fa1f46ecc45366dce3c1b8a5ce18c04de86daf61ac035a4c5c6
(duas execuções seguidas devolveram o mesmo output_sha256)
```

A leitura para um auditor é direta: a primeira parte da afirmação é sustentada por uma única fonte (o resumo cita o regulamento e por isso não conta como segunda confirmação), e a segunda parte é contradita por um trecho localizado do próprio regulamento. Por isso a afirmação inteira é **REFUTED**, pela regra K1 da versão 2.0.0.

7 Camada de stance: regras antes do modelo

A stance é a relação entre uma evidência e uma afirmação: sustenta, contradiz ou é neutra. Ela é um **sinal de entrada** para o Judge, nunca o veredito. O projeto adota uma camada híbrida (ADR-005), na qual comparações exatas são decididas por regras e só o restante vai para um modelo de inferência textual (NLI).

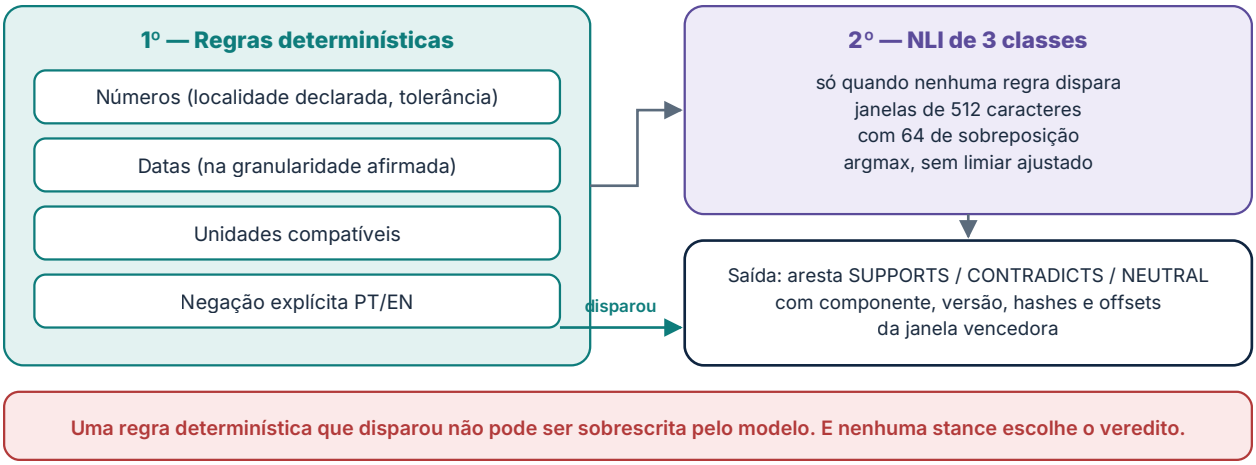


Figura 6. Precedência da camada de stance.

Semântica numérica (ADR-006)

A diferença relativa é $|x - y| / \max(|x|, |y|)$. Há contradição numérica apenas quando o atributo e a unidade coincidem, a leitura do número tem sucesso na localidade declarada da fonte e a diferença excede uma tolerância congelada. Formatação ambígua de localidade falha de forma fechada, em vez de ser reinterpretada.

Caminhos de modelo avaliados

Caminho	Descrição	Licença operacional
A	NLI multilíngue direto	EVAL_ONLY (treino declarado inclui XNLI)
B	Traduzir o lado em português para inglês e verificar	Modelo de tradução EVAL_ONLY por precaução, até concluir o inventário de licenças OPUS; modelo NLI classificado COMMERCIAL_DEFAULT no registro de licenças (2026-10-09)
C	Somente regras (ablação)	sem modelo

Uma **política de licenças de modelos** impede que um modelo marcado como EVAL_ONLY seja usado em operação comercial: um *license guard* no código rejeita esses backends fora do modo de avaliação, salvo opt-in explícito registrado na proveniência. As classificações são de política interna, não parecer jurídico. O texto traduzido é usado só como entrada do modelo; a trilha de auditoria guarda sempre o trecho original.

8 Rastreabilidade: como tudo é registrado

Rastreabilidade, aqui, significa conseguir responder, para qualquer veredito: *qual entrada, quais evidências, quais trechos, quais regras, qual versão do software e qual resultado*, e conferir que nada disso foi alterado depois.

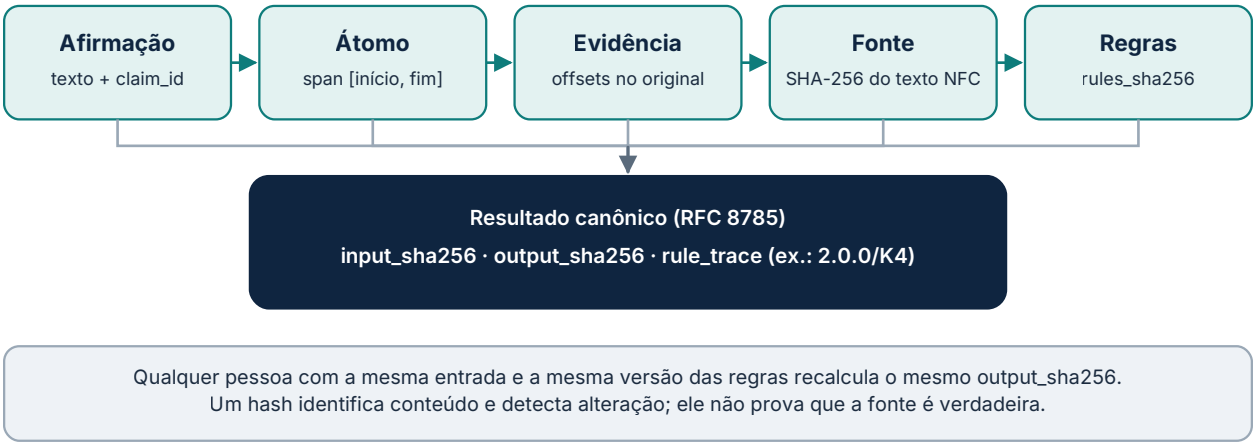


Figura 7. A trilha de prova liga cada elemento ao resultado por meio de identificadores e hashes.

Camadas de rastreabilidade e seu estado

Camada	O que registra	Estado
Saída canônica do Judge	rules_sha256, input_sha256, output_sha256 e rule_trace com o ID de cada regra aplicada (ex.: `2.0.0/K4`). Serialização RFC 8785 testada byte a byte.	IMPLEMENTADO
Evidências admitidas e excluídas	Para cada átomo: evidências que contaram e as que foram rejeitadas, com motivo.	IMPLEMENTADO
Trilha de auditoria operacional	SQLite versionado com migração idempotente; `GET /v1/audit/{trace_id}` informa `replay_integrity` válido ou inválido. Não persiste prompts, trechos ou credenciais.	IMPLEMENTADO
Replay por conteúdo	`ReplayStore` aceita só digests hexadecimais de 64 caracteres e rejeita arquivo cujo digest interno difere do pedido.	IMPLEMENTADO
Logs de acesso	Logs estruturados com cuidado de privacidade sob Uvicorn.	IMPLEMENTADO
Modelo de proveniência W3C PROV-O	Mapeamento de entidades, atividades e agentes para reconstruir como cada resultado foi produzido.	PROPOSTO
Log de transparência Merkle (ADR-010)	Cada decisão vira um evento numa árvore de Merkle com checkpoints assinados, provas de inclusão e testemunhas.	PARCIAL
Carimbo do tempo ICP-Brasil	Carimbo RFC 3161 periódico sobre checkpoints, emitido por ACT credenciada.	PROPOSTO

LIMITE RECONHECIDO PELO PRÓPRIO PROJETO

A trilha atual em SQLite **não é à prova de adulteração** contra quem administra o banco: quem tem acesso pode alterar uma linha e seu digest juntos. Isso está documentado como lacuna R1 no modelo de ameaças e é o que a ADR-010 pretende resolver quando implantada. Em qualquer camada, um hash identifica conteúdo e detecta alteração; ele não autentica autoria, data ou veracidade.

O registro de auditoria verificável (ADR-010)

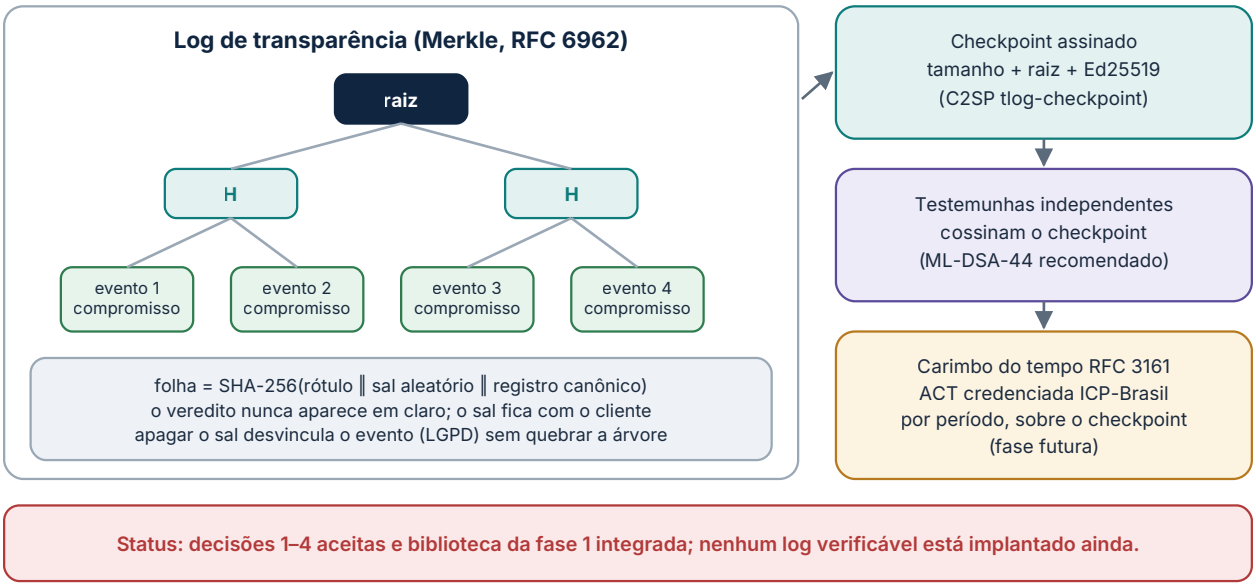


Figura 8. Desenho aprovado do log de transparência: árvore de Merkle no formato C2SP tlog-tiles, checkpoints assinados, testemunhas e carimbo do tempo.

Fase	Entrega	Estado
1	Biblioteca: árvore RFC 6962, provas de inclusão e consistência, compromisso com sal e vetores de teste	IMPLEMENTADO
2	Checkpoints assinados (Ed25519), armazenamento em tiles, escritor único, testes de mutação	PENDENTE
3	Gravação de `/v1/verify`, recibos, `Idempotency-Key`, alteração do OpenAPI	PENDENTE
4	Estado externo: testemunhas, carimbos RFC 3161 e ancoragem	PENDENTE

Por que Merkle e não uma simples corrente de hashes. Numa corrente, provar um evento ou provar que o log só cresceu exige a corrente inteira. Numa árvore de Merkle, uma prova de inclusão usa cerca de $\log_2(n)$ hashes, e a prova de consistência entre dois estados é compacta. A corrente foi o primeiro rascunho e foi rejeitada por isso.

Privacidade. Como o Judge é determinístico, quem adivinha a entrada pode calcular a saída. Publicar `input_sha256` diretamente exporia entradas curtas a ataques de dicionário. Por isso cada folha contém apenas um **compromisso com sal aleatório de 32 bytes**. O veredito nunca aparece em claro. Apagar o sal e o registro torna o evento desvinculável, atendendo a pedidos de eliminação (LGPD), sem quebrar a árvore.

Falha fechada. No modo de registro obrigatório, se o evento não puder ser gravado, a API não devolve veredito. Nenhum veredito circula sem registro.

Defesa contra o próprio operador. Sem estado fora do controle do operador, nenhuma garantia existe contra quem controla aplicação, banco, backups e chave. Por isso a defesa real vem de fora: recibos guardados pelo cliente, cossinaturas de testemunhas independentes, carimbos do tempo ICP-Brasil e ancoragem em logs públicos.

9 Serviços: o que cada um faz e como é rastreado

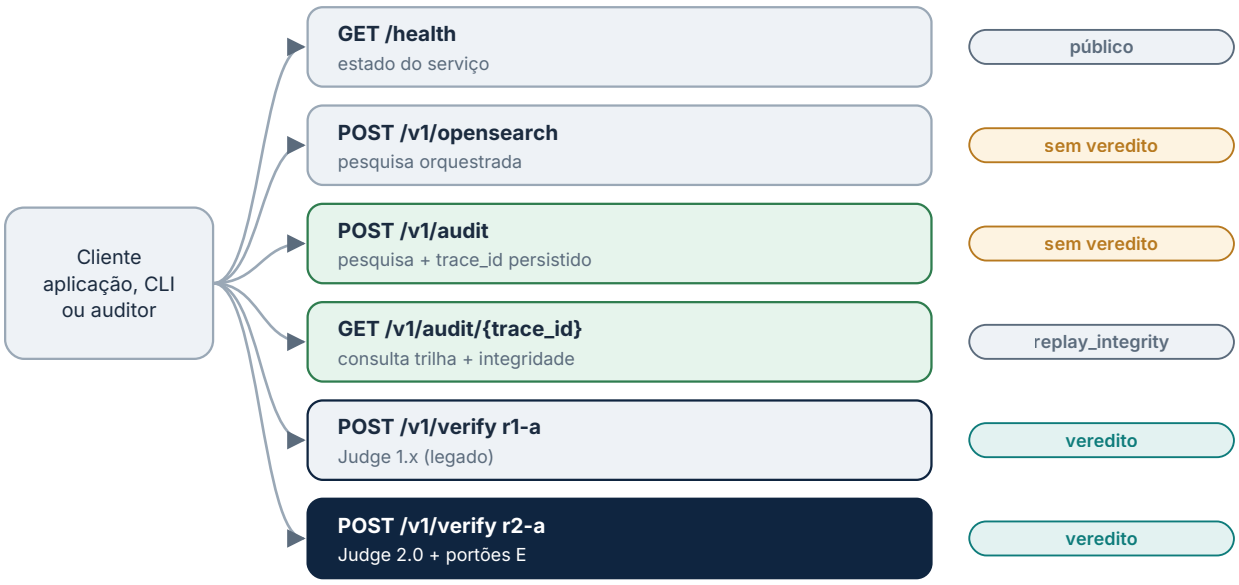


Figura 9. Mapa de serviços da API. Só /v1/verify emite veredito factual.

Serviço	O que faz	Como se comporta e o que garante	Rastro
GET /health	Informa se o serviço está no ar. É a única rota sem chave; as demais exigem chave no modo padrão.	Não registra nem decide nada.	—
POST /v1/opensearch	Executa a pesquisa orquestrada com os provedores configurados e devolve resposta, evidências e falhas.	Resposta com `verification = null` e `verdict_authority = not_adjudicated`. Uma pesquisa nunca é confundida com uma verificação.	logs de acesso
POST /v1/audit	Mesma base operacional, com achados de auditoria e um `trace_id`.	Persiste um resumo da trilha (sem prompts, trechos ou credenciais) em SQLite versionado.	trace_id + digest de replay
GET /v1/audit/{trace_id}	Consulta uma trilha gravada.	Recalcula e informa `replay_integrity` (válido/inválido).	trilha + integridade
POST /v1/verify (r1-a)	Judge 1.x legado, preservado sem alteração para compatibilidade.	Contrato de fio congelado, testado byte a byte.	hashes do resultado
POST /v1/verify (r2-a)	Judge 2.0: portões G e E, regras A, R e K, conferência de trechos contra `source_texts`.	Resultado canônico com rules/input/output SHA-256 e rule_trace. Futuro: recibo e prova de inclusão (ADR-010).	hashes + rule_trace

CLI `search`	Pesquisa pela linha de comando, via orquestrador.	Mesmo comportamento sem veredito.	—
CLI `eval`	Avaliação reproduzível sobre conjuntos Golden congelados.	Relatório JSON com commit, versão do conjunto e digest semântico.	SHA-256 do relatório

Exemplo de requisição ao Judge 2.0

```
POST /v1/verify
{ "schema_version": "r2-a",
  "claim_id": "c1",
  "claim_text": "O regulamento X fixa prazo de 30 dias ...",
  "atoms":    [{ "id": "a1", "text": "prazo de 30 dias para resposta",
                 "start_offset": 21, "end_offset": 51 }],
  "source_texts": { "doc1": "Art. 5º A prestadora deve responder ..." },
  "evidence": [{ "id": "e1", "claim_id": "c1", "source_id": "doc1",
                 "source_location": "doc1#art5", "retrieved_at": "2026-10-18T10:00:00Z",
                 "start_offset": 21, "end_offset": 50,
                 "excerpt": "deve responder em até 30 dias",
                 "origin_id": "regulamento-x",
                 "normalized_content_hash": "",
                 "provenance_complete": true }],
  "stances":  [{ "id": "s1", "atom_id": "a1", "evidence_id": "e1", "stance": "SUPPORTS" }],
  "out_of_scope": false, "factual_claim": true, "temporal_anchor_present": true,
  "retrieval_succeeded": true, "evidence_pool_integrity_ok": true }
```

Campos conforme `Judge2VerificationRequest` em `verification/api_models.py` e `docs/api/openapi.json`. Uma requisição avalia uma afirmação; os sinalizadores de abstenção têm os valores padrão mostrados. A resposta traz `result` (saída canônica), `simple_explanation` (até 5 linhas) e `technical_explanation`.

Política de resposta separada do veredito

Uma aplicação pode decidir, por exemplo, não exibir ao usuário uma resposta cujo veredito seja `UNVERIFIED`. Essa escolha fica num campo próprio (`response_policy`), separado de `decision_state` e `verdict`, para que a política de produto nunca se confunda com a decisão factual.

10 Aplicações por setor

O padrão é sempre o mesmo: uma resposta produzida com IA é decomposta em afirmações, cada afirmação é verificada contra um conjunto de evidências autorizado, e o resultado fica registrado de forma reproduzível. O que muda de setor para setor é a fonte de verdade e o que precisa ser preservado.

Setor	O que se verifica	Fonte de verdade	O que fica rastreado
Judiciário e advocacia	Precedentes e citações em peças, minutas e decisões.	Texto oficial do acórdão ou da norma; posição exata do trecho.	Hashes, regra aplicada, recibo juntável aos autos.
Setor público	Respostas de assistentes a cidadãos; pareceres internos.	Normas, atos e bases oficiais.	Trilha para controle interno, ouvidoria e tribunais de contas.
Telecomunicações	Respostas sobre direitos do consumidor.	Regulamento (ex.: RGC, Res. Anatel 765/2023).	Afirmação, dispositivo e versão capturada da norma.
Serviços financeiros	Explicações de IA a clientes; relatórios de compliance.	Políticas aprovadas, normativos, contratos.	Evidência da política usada em cada resposta.
Mercado de capitais	Resumos de relatórios e fatos relevantes.	Documentos publicados pela companhia.	Preservação de números e datas (regras numéricas).
Seguros	Respostas sobre coberturas e exclusões.	Texto da apólice e das condições gerais.	Qual cláusula sustenta ou contradiz a resposta.
Saúde	Resumos clínicos e respostas a protocolos.	Protocolos e bulas oficiais.	Conflitos explícitos entre fontes (CONFLICTING).
Auditoria interna	Conclusões assistidas por IA em trabalhos de auditoria.	Papéis de trabalho e documentos de suporte.	Registro reproduzível de cada conclusão.
Pesquisa e educação	Revisões de literatura e respostas a estudantes.	Artigos e materiais de referência.	Citações conferidas por trecho.

APLICAÇÕES POTENCIAIS, NÃO IMPLANTAÇÕES

A tabela descreve onde a arquitetura pode ser aplicada. Nenhuma dessas aplicações está em produção hoje, e qualquer implantação regulada exige análise jurídica, avaliação de impacto, controles de acesso, política de retenção e validação independente.

Por que telecomunicações foi escolhido como primeiro domínio

O benchmark em português começa pelo **Regulamento Geral de Direitos do Consumidor de Serviços de Telecomunicações (RGC)**, aprovado pela Resolução Anatel nº 765, de 6 de novembro de 2023. É um texto oficial, público, estruturado em dispositivos, com prazos, direitos, obrigações e exceções, exatamente os tipos de afirmação que uma IA costuma errar. A escolha delimita o corpus de avaliação; **não indica parceria, apoio ou validação da Anatel**.

Situação normativa na página oficial consultada em 18/10/2026 (atualizada pela Anatel em 06/07/2026): a maior parte do regulamento vigora desde 1º/09/2025, com as exceções do Acórdão nº 228/2024; os arts. 2º a 79 da resolução, que alte-

ravam outros regulamentos, constam como revogados pela Resolução nº 777/2025. O benchmark usa a versão capturada e registrada por SHA-256 (#177) e não interpreta vigência, hierarquia ou aplicabilidade de dispositivos.

11 Processos judiciais e prova

Num processo, o que importa não é só se uma afirmação está certa, mas **se é possível demonstrar** por que está certa ou errada, de forma que a outra parte e o juízo possam conferir. O ATTRACTOR foi desenhado para produzir esse tipo de registro.

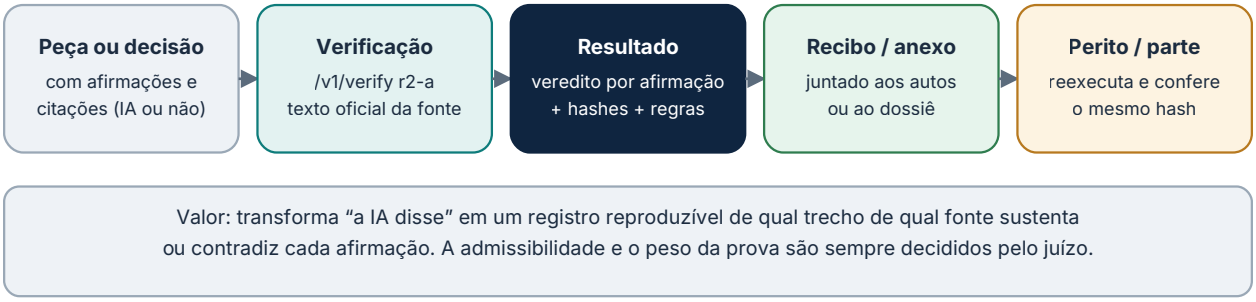


Figura 10. Fluxo de uso em contexto judicial: a verificação gera um registro que a outra parte ou um perito podem reexecutar.

Conexão com o ordenamento brasileiro

Norma	Conteúdo (paráfrase)	Como o ATTRACTOR pode apoiar
CPC, art. 369	As partes podem usar todos os meios legais e moralmente legítimos para provar a verdade dos fatos.	O resultado reproduzível pode compor elementos de prova técnica; o peso é do juízo.
CPC, arts. 77 a 81	Deveres das partes e litigância de má-fé.	Verificar citações antes de protocolar reduz o risco de peças com precedentes inexistentes.
CPP, arts. 158-A a 158-F	Cadeia de custódia dos vestígios.	Por analogia técnica: registro de origem, integridade (hash) e sequência de manuseio do dado.
LGPD, art. 20	Direito de solicitar revisão de decisões tomadas unicamente com base em tratamento automatizado.	A trilha mostra o que foi decidido e com base em quê, apoiando pedidos de revisão.
Res. CNJ 615/2025, art. 27	O sistema deve impedir a alteração dos dados recebidos antes do uso no desenvolvimento de soluções de IA, com controle de versões, tokens e registros de auditoria; o dispositivo trata dos dados e datasets de desenvolvimento.	Por analogia de prática, não de obrigação: regras e artefatos versionados com hash. O ATTRACTOR não é um sistema de gestão de datasets.
Res. CNJ 615/2025, art. 13, V, e art. 21, §2º	Registro automático da operação (log), sempre que tecnicamente possível; produtos gerados por IA devem registrar o uso de IA nos logs, com rótulos, para estatística, monitoramento e auditoria.	Aplicação potencial: o registro por evento previsto na ADR-010 (ainda não implantado) poderia apoiar esse tipo de log.

Res. CNJ 615/2025, art. 10, I, e art. 34	Vedação de soluções que impeçam revisão humana; magistrado pode modificar a saída da IA.	O veredito é explicável por regra e não substitui a decisão humana.
Res. CNJ 615/2025, art. 4º, XVII e XVIII	Definições de auditabilidade e explicabilidade.	Explicação simples e técnica para cada resultado (ADR-009 §13).

AVISO JURÍDICO

Este capítulo é técnico e informativo. Não é parecer jurídico. A admissibilidade, a validade e o peso de qualquer elemento de prova são decididos pelo juízo competente. A Resolução CNJ nº 615, de 11 de março de 2025, dispõe sobre soluções de IA no Poder Judiciário; a Resolução CNJ nº 674, de 25 de março de 2026, alterou apenas o seu art. 15 (composição do comitê), que não é citado aqui. O uso do ATTRACTOR não estabelece conformidade com nenhuma dessas normas. Os dispositivos da Resolução 615 e o art. 20 da LGPD foram conferidos no texto oficial em 18/10/2026; os textos do CPC (art. 369) e do CPP (arts. 158-A a 158-F) não puderam ser conferidos na página oficial nesta revisão, por limitação técnica de acesso, e devem ser conferidos antes de qualquer uso jurídico.

Rastreabilidade para qualquer setor

O raciocínio vale fora do Judiciário. Em qualquer atividade regulada (bancos, seguradoras, operadoras, órgãos públicos), a pergunta de um auditor, de um regulador ou de um cliente é a mesma: *em que você se baseou?* Um registro que liga cada afirmação ao trecho da fonte, à regra aplicada e a um hash reproduzível responde a essa pergunta sem depender da memória de ninguém nem da boa vontade de um fornecedor de IA.

12 Por que isso importa para a auditoria de IA no Brasil

1. A regulação está chegando, e ela pede rastro

O **PL 2.338/2023**, o Marco Legal da IA, foi aprovado no Senado e tramita na Câmara dos Deputados, na Comissão Especial sobre Inteligência Artificial, aguardando parecer do relator (consulta em outubro de 2026).[5] No Judiciário, a **Resolução CNJ nº 615/2025** já exige controle de versão, registros de auditoria, explicabilidade e revisão humana. A LGPD já garante o direito de pedir revisão de decisões automatizadas. Na Europa, o Regulamento (UE) 2024/1689 trata de registros e supervisão humana. Todas apontam na mesma direção: **não basta a IA acertar; é preciso demonstrar como chegou ao resultado**.

2. O português e o direito brasileiro ficaram para trás

A maior parte das ferramentas e dos benchmarks de verificação de alucinação é construída e medida em inglês. O ATTRACTOR trata o português como alvo primário de avaliação: tem regras de negação PT/EN, semântica numérica sensível à localidade (vírgula decimal, por exemplo) e um benchmark sobre texto normativo brasileiro em preparação.

3. Independência de fornecedor

O código é aberto (Apache-2.0), o Judge 2.0 executa sem acesso à rede (teste T9 da ADR-009) e não depende de nenhum provedor de modelo para decidir. Um órgão público ou uma empresa podem auditar o próprio auditor: ler as regras, rodar os testes e recalcular os hashes.

4. Uma alternativa ao “LLM julgando LLM”

Abordagem	Como decide	Limite para auditoria
Segundo modelo como juiz	Outro LLM avalia a resposta do primeiro.	Não é reproduzível nem explicável por regra; pode alucinar sobre a alucinação.
Escores de groundedness	Modelo especializado devolve um número de 0 a 1.	Útil para triagem, mas um número não diz qual trecho sustenta qual afirmação.
AYORAI ATTRACTOR	Modelos geram sinais; regras públicas e versionadas decidem.	A qualidade depende da etapa de stance, que ainda precisa ser medida em português.

LEITURA HONESTA DO POSICIONAMENTO

Plataformas comerciais estabelecidas já oferecem detecção de alucinação e checagem de groundedness, e grandes provedores de nuvem incluem recursos semelhantes. O diferencial do ATTRACTOR não é detectar mais alucinações; é produzir um **registro auditável e reproduzível** de cada decisão, com foco no português e no contexto regulatório brasileiro. Se esse registro tem valor maior que um escore depende de resultados que o projeto ainda precisa demonstrar.

13 Segurança e modelo de ameaças

O modelo de ameaças (`docs/THREAT-MODEL.md`) protege a integridade da verificação: afirmações fornecidas pelo chamador, evidências em mundo fechado, stance derivada de evidência, proveniência preservada e veredito escolhido pelo Judge.

ID	Ameaça	Cenário	Defesa atual	Lacuna
A	Prompt injection	Texto de documento tenta mudar o veredito.	Afirmação como entrada; veredito só pelo Judge; casos de injeção no Golden.	—
B	Citação forjada	IA fornece trecho que não existe na fonte.	Trecho localizado no original após normalização NFC; evidência não encontrada é descartada e registrada.	—
C	Proveniência errada em tradução	Posições do texto traduzido gravadas como se fossem do original.	Proveniência da tradução separada; trilha guarda o trecho original.	alinhamento por frase
D	Adulteração de replay	Arquivo de replay alterado ou caminho manipulado.	Digest de 64 hex obrigatório; rejeição se o digest interno diferir.	—
E	Vazamento do Golden v1	Casos ocultos usados em desenvolvimento.	Padrões de .gitignore e teste de isolamento.	**alta**: sem controle técnico de acesso
F	Contaminação de licença	Modelo EVAL_ONLY usado comercialmente.	*License guard* em runtime; script de manifesto de treino.	**alta**: sem portão automático global
G	Sobreajuste à avaliação	Ajustar regras olhando o conjunto visível.	Conjuntos congelados com teste que falha se mudarem; pré-registro.	**média**: controle procedimental
H	Cadeia de suprimentos	Dependência ou Action comprometida.	Auditoria de dependências, Bandit, CodeQL, Actions fixadas por SHA, Dependabot.	—
R1	Trilha adulterável	Administrador altera linha e digest.	Desenho ADR-010 (Merkle, testemunhas, carimbo).	**alta**: não implantado

Controles de serviço. Autenticação por chave ativa por padrão, mas desativável por configuração explícita; limite configurável de tamanho do corpo; `/health` público. **Não há** limitação de taxa nem TLS próprio do serviço: a terminação TLS e a proteção contra abuso dependem da infraestrutura de implantação. O serviço não deve ser descrito como pronto para produção.

Também estão fora de escopo, por declaração explícita: provar que a fonte externa é verdadeira, proteger o host ou a conta do GitHub, impedir que um modelo ou dependência comprometidos ajam de forma maliciosa e garantir generalização além dos dados avaliados.

14 Avaliação científica e estado atual dos resultados

O projeto aplica a si mesmo o mesmo padrão de verificabilidade que propõe para a IA: conjuntos de avaliação congelados por hash, pré-registro antes de medir, intervalos de confiança, testes estatísticos pareados e divulgação dos resultados ruins.

Resultados de desenvolvimento publicados

Os números abaixo foram medidos com as regras do **Judge 1.x** (seis estados, incluindo VERIFIED), na avaliação F1 do commit `c4b73f4` (03/10/2026) para o Golden v0, e na auditoria E1 para o Golden v0.1. Ainda não há resultado do Judge 2.0 sobre os conjuntos Golden.

Caminho	Golden v0	Golden v0.1
A — NLI multilíngue (EVAL_ONLY)	40,00%	76,67%
B — traduzir e verificar	43,33%	66,67%
C — só regras	26,67%*	33,33%
Baseline de classe majoritária	**43,33%**	**43,33%**

IC95% por bootstrap no Golden v0 (F1): A [23,33%; 56,67%]; B [26,67%; 60,00%]; C [13,33%; 43,33%]. *O caminho C mediu 26,67% no commit c4b73f4; com o detector de regras atual, o mesmo caminho mede **23,33%** no Golden v0 (F1.2-RESULTS). Os valores originais são preservados como registro histórico.

COMO LER ESSES NÚMEROS

- São **30 casos** sobre **52 documentos sintéticos em inglês**. Não são benchmark de produção.
- No Golden v0, os intervalos de confiança de 95% dos caminhos A e B **incluem a baseline**: não há evidência de que superem um classificador trivial.
- O Golden v0.1 foi **reescrito depois que o autor viu os resultados por caso**, fato declarado no próprio pré-registro. Seus números não são evidência confirmatória.
- **Correção da v1.0:** a v1.0 afirmava que cerca de 60% bastariam para uma melhora significativa. Isso não se sustenta. No teste exato de McNemar bilateral contra a baseline (13 acertos em 30), 18/30 = 60% dá p = 0,0625 mesmo no melhor arranjo. O mínimo possível é 19/30 (63,3%), e só se o sistema acertar todos os 13 casos que a baseline acerta; no pior arranjo, são necessários 24/30 (80%). A significância depende dos pares discordantes, não só da acurácia (cálculo reproduzível no relatório de auditoria).
- **Não existe resultado em português.**

O benchmark em português (RGC)

O próximo resultado relevante será medido sobre o texto oficial do RGC, com um protocolo desenhado para não permitir que o autor "ajuste" o resultado.



Figura 11. Caminho até o primeiro resultado em português. Cada etapa é um portão: a seguinte só começa com a anterior aprovada.

Estado do protocolo: **PROPOSTO** — não congelado. Nada abaixo é resultado; é o desenho registrado no pré-registro e nas decisões do mantenedor.

- **Hipótese H1:** o sistema supera a baseline de classe majoritária no conjunto oculto de avaliação, definido e anotado antes da execução.
- **Baseline primária (DEC-014, Emenda 5):** a classe majoritária determinada só pelos rótulos da partição pública de desenvolvimento (70%), registrada antes de qualquer execução no conjunto oculto. A classe majoritária da própria partição avaliada é reportada apenas como referência descritiva, nunca como base do teste.
- **Critério primário:** H1 só recebe suporte se, ao mesmo tempo, (1) a acurácia do sistema superar a da baseline em pelo menos 10 pontos percentuais, (2) o limite inferior do IC95% da acurácia ficar acima da acurácia da baseline e (3) o teste exato de McNemar bilateral contra a baseline tiver $p < 0,05$. Acurácia balanceada e macro-F1 são métricas secundárias obrigatórias.
- **Composição:** meta de 300 casos, cerca de um terço composto (com átomos), nas categorias prazos e datas, direitos do consumidor, obrigações da prestadora e exceções.
- **Isolamento:** 70% público para desenvolvimento e 30% oculto, guardado fora do repositório público; só tamanho e hash do conjunto oculto são divulgados.
- **Qualidade dos rótulos:** uma segunda pessoa rotula os mesmos casos sem ver os rótulos da primeira; calcula-se o kappa de Cohen entre os dois anotadores e só com kappa ≥ 0,6 o conjunto pode ser promovido a v1. Marcadores de ambiguidade (RULE_GAP) do piloto não entram no kappa. Divergências são resolvidas em conjunto e registradas caso a caso.
- **Execução única:** o sistema roda uma vez, depois dos rótulos humanos completos e do congelamento do protocolo, sem ajuste de regras, limiares, modelos ou casos em resposta aos resultados. Resultado negativo ou inconclusivo também é publicado.
- **Métricas:** acurácia, acurácia balanceada, macro-F1, intervalos de 95% por bootstrap com 10.000 reamostragens e semente fixa 20261003.

O primeiro piloto exploratório, de 20 casos, foi concluído em 16/10/2026 e **descartado**: serviu para melhorar o guia de anotação (por exemplo, diferenciar “não confirmado” de “contradito”) e o sistema não foi executado sobre ele.

15 Engenharia e governança



Figura 12. Linha do tempo das releases e decisões principais (2026).

1.057	~9,0 mil	~10,1 mil	903
testes aprovados localmente no commit ae44c23	linhas de código-fonte Python	linhas de testes	commits na main em 18/10/2026

- **Integração contínua:** Python 3.11, 3.12 e 3.13, regressão do Golden, regressão do Judge e segurança são verificações obrigatórias na branch `main`, que é protegida. No commit ae44c23 todas passaram no GitHub Actions. Na conferência local, 1 teste falhou por diferença de ambiente (o teste de namespace espera o caminho `site-packages` e o Debian usa `dist-packages`); o job equivalente passou no CI.
- **Segurança da cadeia:** CodeQL, Bandit, auditoria de dependências, Dependabot e todas as GitHub Actions fixadas por SHA completo.
- **Decisões registradas:** ADRs numeradas e decisões do mantenedor (DEC-xxx) com data e justificativa; mudança de regra exige nova versão, nunca sobrescrita.
- **Artefatos congelados:** regras, Golden, manifestos, hashes, capturas de fonte e pré-registros são protegidos por testes que falham se forem alterados.
- **Arquivamento independente:** releases depositadas no Zenodo (CERN) com DOI; o hash do commit liga cada versão ao seu conteúdo exato.
- **Status como fonte única:** `docs/STATUS.md` é a única fonte de estado atual; documentos históricos dizem qual SHA auditaram.

Autoria e uso de ferramentas de IA

O projeto foi criado e é dirigido tecnicamente por Anderson Leon Ayora. Ferramentas de IA foram usadas como assistentes de implementação, revisão e pesquisa, sob direção e revisão do autor. Em 13/10/2026, dos 823 commits da `main`, 813 tinham Anderson como autor, 3 eram atualizações automáticas de dependências e 7 carregavam atribuição técnica a uma ferramenta de IA. Esses 7 foram preservados sem reescrita, porque reescrever o histórico quebraria a integridade verificável das versões arquivadas com DOI. Arquitetura, critérios de aceitação e aprovação de cada mudança são do autor.

> **GOVERNANÇA AINDA EM AMADURECIMENTO**
>
> O projeto tem hoje um único mantenedor. A decisão DEC-013 registra expressamente que, num ciclo de execução, houve dispensa de revisão humana independente, e que isso **não** equivale a uma revisão independente ter ocorrido (achado F-01 em aberto). Trazer revisores independentes é prioridade do plano.

16 O que existe e o que ainda falta

Item	Estado
Contrato Judge 2.0 (ADR-009) com regras G, E, A, R, K	IMPLEMENTADO
Rota /v1/verify r2-a com conferência de trechos contra textos-fonte	IMPLEMENTADO
Contrato legado r1-a congelado	IMPLEMENTADO
Serialização canônica RFC 8785 e hashes de entrada, saída e regras	IMPLEMENTADO
Trilha de auditoria SQLite com replay_integrity	IMPLEMENTADO
Autenticação por chave ativa por padrão (desativável explicitamente)	IMPLEMENTADO
Limite de tamanho do corpo da requisição	IMPLEMENTADO
Stance híbrido (regras + NLI) e pipeline de afirmações	BIBLIOTECA
Biblioteca Merkle da ADR-010 (fase 1)	BIBLIOTECA
Extração de afirmações e inferência de stance dentro da API	PENDENTE
Log de transparência implantado (ADR-010 fases 2 a 4)	PENDENTE
Proveniência PROV-O emitida pelo software	PROPOSTO
Benchmark RGC em português	EM CURSO
Segundo anotador e revisão humana independente	PENDENTE
Limitação de taxa e TLS próprio do serviço	PENDENTE
Isolamento técnico do Golden v1 e portão de licença de treino	PARCIAL
Certificação, validação externa ou cliente em produção	PENDENTE

Esta matriz resume `docs/STATUS.md` e `docs/TRACEABILITY-MASTER.md` em 18/10/2026. Em caso de divergência, valem os arquivos do repositório no commit mais recente.

17 Planos de melhoria contínua

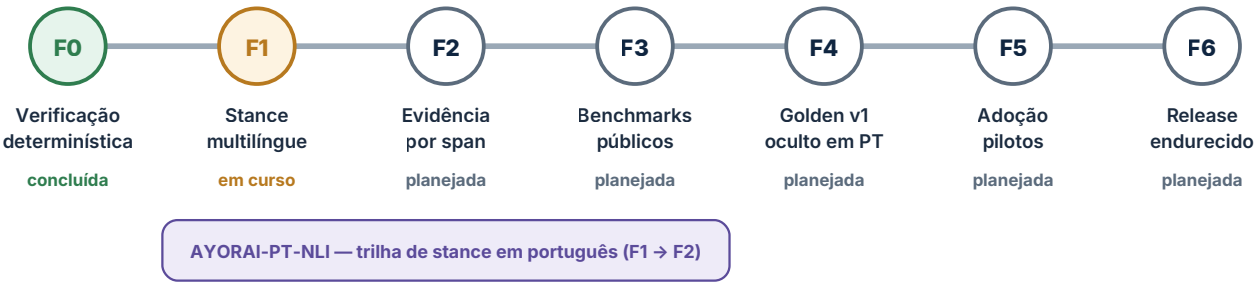


Figura 13. Roteiro F0 a F6. O Judge determinístico permanece fora do modelo em todas as fases.

Plano de 12 meses

Período	Metas	Critério de conclusão
T1	Segundo piloto humano isolado; revisão dos achados; congelamento explícito do protocolo; redação dos 300 casos e rótulos humanos completos.	Nenhuma execução do sistema antes dos portões de protocolo e rotulagem.
T2	Segundo anotador; cálculo de concordância; resolução documentada de divergências; conjunto v1 e relatório.	Kappa ≥ 0,6 antes de chamar o conjunto de v1; caso contrário, documentar e não reclassificar.
T3	Melhorar o detector de stance em português (AYORAI-PT-NLI), respeitando a política de licenças.	Nenhuma alteração retroativa do benchmark congelado; nova avaliação claramente identificada.
T4	Demonstração reproduzível e publicação técnica.	Outra pessoa consegue reproduzir; texto separa resultado de hipótese.

Linhas de melhoria técnica

- **Fábrica de dados jurídicos por construção.** Casos derivados de texto normativo com proveniência desde a origem.
- **Stance em português sobre encoder brasileiro.** Modelo de NLI treinado só com dados de licença comercial verificada.
- **Verificação temporal.** Conferir afirmações contra a versão da norma vigente na data relevante, como metadado, sem interpretar vigência.
- **Justificativas citadas.** Explicação técnica que aponta trecho por trecho a base de cada veredito.
- **Abstenção conformal.** Usar incerteza calibrada para se abster, nunca para escolher veredito.
- **Log de transparência dos vereditos.** Concluir as fases 2 a 4 da ADR-010, com testemunhas independentes e carimbo do tempo ICP-Brasil.
- **Integração da extração à API.** Oferecer o fluxo completo (resposta → afirmações → evidências → stance → Judge) como serviço.

O que o projeto busca

- **Pilotos** com organizações que queiram testar a verificação em fluxos controlados e com dados autorizados.
- **Anotadores** especialistas, principalmente em direito do consumidor e regulação, para o benchmark em português.
- **Revisores técnicos independentes** para código, protocolo e segurança.
- **Fomento** para pesquisa aplicada, anotação, custódia do conjunto oculto e infraestrutura.

18 Como participar, citar e usar

Executar localmente

```
git clone https://github.com/Ayorinha/ayorai-opensearch.git
cd ayorai-opensearch
python -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"
pytest                                  # suíte completa
attractor eval --suite golden-v0 --out reports/golden-v0.json
```

Ao registrar qualquer resultado, anote sempre o SHA do commit, a versão do conjunto e o digest do relatório.

Contribuir

Contribuições exigem DCO (*Developer Certificate of Origin*). Mudanças em regras, conjuntos congelados ou contratos exigem decisão versionada do mantenedor. Erros e falsos resultados devem ser relatados como *issues* no repositório.

Citar

Para todas as versões: DOI **10.5281/zenodo.23125082**. Versões específicas: v0.5.0, DOI 10.5281/zenodo.23344005; v0.4.0, DOI 10.5281/zenodo.23300849. Metadados completos em `CITATION.cff`.

Licença e marca

O repositório é distribuído sob a **Apache License 2.0** (arquivo `LICENSE`). A licença concede permissões de uso, mas não transfere titularidade: Anderson Leon Ayora permanece titular dos direitos sobre a obra que criou, e contribuições de terceiros continuam pertencendo a seus autores, licenciadas nos mesmos termos e sujeitas ao DCO. Pela seção 4(d), quem distribuir obras derivadas deve incluir uma cópia legível dos avisos de atribuição do arquivo `NOTICE`. "AYORAI" e "AYORAI ATTRACTOR" são usados como marcas do autor (`TRADEMARKS.md`); não há pedido de registro no INPI registrado no repositório, e a Apache-2.0 não concede direito de uso de marcas (seção 6). Este parágrafo descreve a licença; não é parecer jurídico.

> **DECLARAÇÃO DE INDEPENDÊNCIA**
>
> O AYORAI ATTRACTOR é um projeto independente. Nenhuma menção neste documento a Anatel, CNJ, OAB, tribunais, universidades ou empresas indica parceria, apoio, endosso ou validação dessas instituições.

19 Glossário

Termo	Definição
Afirmação (claim)	Frase factual a ser verificada.
Átomo	Parte mínima de uma afirmação composta, apontada por posição na frase.
Evidência	Trecho de um documento-fonte, com posições (offsets) e identificação da fonte.
Stance	Relação entre evidência e átomo: SUPPORTS, CONTRADICTS ou NEUTRAL.
Judge	Componente determinístico que aplica as regras e emite o veredito; o mesmo resultado canônico para a mesma entrada estruturada e as mesmas regras.
Veredito	SUPPORTED, PARTIALLY_SUPPORTED, REFUTED, UNVERIFIED ou CONFLICTING.
Estado de decisão	DECIDED, ABSTAINED ou INVALID_INPUT; só DECIDED tem veredito.
Cluster	Grupo de evidências com dependência conhecida entre si, contado como uma fonte. Clusters distintos não provam independência.
Robustez	R0 a R3: força e independência da evidência, sem alterar o veredito.
rule_trace	Lista das regras aplicadas, com identificador e versão.
SHA-256	Função de hash que gera uma impressão digital de 64 caracteres hexadecimais; qualquer alteração no conteúdo muda o hash.
RFC 8785	Padrão de serialização canônica de JSON, para que implementações diferentes produzam os mesmos bytes.
NFC	Forma de normalização Unicode usada para comparar trechos sem diferenças invisíveis de codificação.
NLI	Inferência de linguagem natural: classificação automática da relação entre duas frases.
Golden	Conjunto de casos de avaliação com rótulos de referência, congelado por hash.
Árvore de Merkle	Estrutura de hashes que permite provar que um evento faz parte de um registro sem revelar o registro inteiro.
Checkpoint	Estado assinado de um log de transparência: tamanho e raiz da árvore.
Testemunha	Parte independente que cossina checkpoints, dificultando que o operador reescreva o histórico.
Carimbo do tempo	Prova emitida por terceiro credenciado de que um dado existia numa data (RFC 3161; ICP-Brasil).
Pré-registro	Publicação do método de avaliação antes de medir, para impedir ajuste posterior.
Kappa de Cohen	Medida de concordância entre dois anotadores que desconta o acaso.

Histórico de revisões deste documento

Versão	Data	Alterações

v1.0	18/10/2026	Primeira versão, substituída por esta; o arquivo original continua acessível no histórico do repositório (commit 9be3a38).
v1.1	18/10/2026	Revisão após auditoria contra o código no commit ae44c23 e contra fontes oficiais: correção da afirmação estatística de 60%; delimitação do determinismo ao Judge; exemplo de API corrigido e executado; abstenção informada pelo chamador; SOURCE_UNAVAILABLE; semântica de clusters e robustez; rotulagem dos resultados Golden (Judge 1.x, caminho C atual 23,33%); protocolo RGC com DEC-014; paráfrases da Res. CNJ 615 corrigidas e Res. CNJ 674/2026; vigência do RGC; controles de serviço; licença e titularidade. Detalhes em `docs/institutional/AUDITORIA-DOCUMENTO-TECNICO-v1.1.md`.

20 Referências

Fontes externas citadas no texto

1. UNC Law Library. *Handling Hallucinations: A Reflection on the Last Three Years*, jun. 2026 (Mata v. Avianca; base de Damien Charlotin). https://library.law.unc.edu/2026/06/handling-hallucinations-a-reflection/
2. Lex. *TJPR multa advogado por litigância de má-fé pelo uso de IA "sem zelo e cautela"*, 2026. Processo 0025387-22.2025.8.16.0001. https://www.lex.com.br/tjpr-multa-advogado-por-litigancia-de-ma-fe-pelo-uso-de-ia-sem-zelo-e-cautela/
3. Tecnoblog. *Advogado tenta prompt injection de IA e recebe multa de R$ 32,8 mil*, 2026. https://tecnoblog.net/noticias/advogado-tenta-prompt-injection-de-ia-e-recebe-multa-de-r-328-mil/
4. Convergência Digital. *Justiça denuncia advogado à OAB por mau uso da inteligência artificial*, jan. 2026. https://convergenciadigital.com.br/inovacao/justica-denuncia-advogado-a-oab-por-mau-uso-da-inteligencia-artificial/
5. Câmara dos Deputados. Ficha de tramitação do PL 2.338/2023 (consulta em 18/10/2026). https://www.camara.leg.br/proposicoesWeb/fichadetramitacao?idProposicao=2487262
6. CNJ. Resolução nº 615, de 11 de março de 2025 (conferida em 18/10/2026). https://atos.cnj.jus.br/files/original1555302025031467d4517244566.pdf
7. CNJ. Resolução nº 674, de 25 de março de 2026 (altera o art. 15 da Res. 615). https://atos.cnj.jus.br/files/original1224352026032669c525831ce70.pdf
8. Brasil. Lei nº 13.709/2018 (LGPD), art. 20 conferido em 18/10/2026. https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm
9. Brasil. Lei nº 13.105/2015 (Código de Processo Civil); art. 369 não conferido nesta revisão. https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2015/lei/l13105.htm
10. Brasil. Decreto-Lei nº 3.689/1941 (Código de Processo Penal), arts. 158-A a 158-F; não conferidos nesta revisão. https://www.planalto.gov.br/ccivil_03/decreto-lei/del3689.htm
11. Anatel. Resolução nº 765, de 6 de novembro de 2023 (RGC); página consultada em 18/10/2026. https://informacoes.anatel.gov.br/legislacao/resolucoes/2023/1900-resolucao-765
12. União Europeia. Regulamento (UE) 2024/1689 (AI Act). https://eur-lex.europa.eu/eli/reg/2024/1689/oj
13. ISO/IEC 42001 — Sistemas de gestão de IA. https://www.iso.org/standard/81230.html
14. W3C. PROV-O: The PROV Ontology. https://www.w3.org/TR/prov-o/
15. IETF. RFC 8785 — JSON Canonicalization Scheme. https://datatracker.ietf.org/doc/html/rfc8785
16. IETF. RFC 6962 — Certificate Transparency. https://datatracker.ietf.org/doc/html/rfc6962
17. C2SP. tlog-tiles, tlog-checkpoint, tlog-witness. https://c2sp.org/tlog-tiles
18. ITI. Carimbo do tempo (ICP-Brasil). https://www.gov.br/iti/pt-br/acesso-a-informacao/perguntas-frequentes/carimbo-do-tempo
19. Correio Braziliense. *OAB Nacional aprova recomendações para uso de IA na prática jurídica*, nov. 2024. https://blogs.correiobraziliense.com.br/cbpoder/oab-nacional-aprova-recomendacoes-para-uso-de-ia-na-pratica-juridica/
20. rfp.wiki. *Vectara vs Cleanlab* (mercado de detecção de alucinação). https://www.rfp.wiki/vendors/vectara/cleanlab

Documentos do repositório (github.com/Ayorinha/ayorai-opensearch)

- `README.md` — visão geral, evidências v0.3.0, roteiro
- `docs/STATUS.md` — fonte única do estado atual (18/10/2026)
- `docs/ARCHITECTURE.md` — visão geral da arquitetura
- `docs/manual/01-visao-geral.md e 02-arquitetura.md` — manual técnico
- `docs/adr/ADR-002 a ADR-010` — decisões de arquitetura

- `rules/judge-2.0.0.json` — regras executáveis do Judge 2.0
- `docs/THREAT-MODEL.md` — modelo de ameaças
- `docs/PROVENANCE-MODEL.md` — mapeamento PROV-O
- `docs/CONFORMIDADE-BRASIL.md` — notas de conformidade
- `docs/eval/PREREGISTRATION-RGC-v1.md` — pré-registro do benchmark RGC
- `docs/eval/F1.2-RESULTS.md` — resultados de desenvolvimento
- `docs/institutional/PLANO-DE-TRABALHO-12M.md` — plano de 12 meses
- `docs/legal/PRIOR-IP-RECORD.md` — registro de autoria e anterioridade
- `docs/AUTHORSHIP.md` — autoria e uso de IA

Documento preparado em 18/10/2026 a partir do repositório na branch `main` e de fontes públicas consultadas na mesma data. Revisão v1.1 conferida contra o commit ae44c23: 1.057 testes aprovados localmente, 1 falha de ambiente (teste de namespace; o job equivalente passou no CI), e todas as verificações obrigatórias do GitHub Actions concluídas com sucesso no mesmo SHA.